CREDITI ED ESAMI, A TEMPO!

INDICE

- Presentazione del problema
- Presentazione dell’algoritmo
- Complessità del codice
- Conclusione

Presentazione del problema

- **N** esami
- **T** tempo
- **C** crediti
- **K** crediti massimi

- Ogni esame impiega **T** tempo per poter essere eseguito
- Ogni esame eseguito da **C** crediti allo studente
- Ogni studente può possedere al massimo **K** crediti
- Ogni studente può comprare un esame, spendendo **K** crediti

Scopo dell’algoritmo?

Determinare il tempo minimo necessario a passare tutti gli esami

Input

Num esami	3	4	Num crediti accumulabili
t esame 1	1	1	c esame 1
t esame 2	2	3	c esame 2
t esame 3	4	1	c esame 3

Metodo di risoluzione

- 2 ARRAY:
  - SITUAZIONE DELLO STUDENTE FINO ALL'ESAME X
  - SITUAZIONE DELLO STUDENTE DOPO L'ESAME X

- LUNGHEZZA DEGLI ARRAY = NUMERO DI CREDITI MASSIMO (K + 1)

- CELLA = SOMMA DEI TEMPI DEGLI ESAMI SVOLTI FINO ALL'ESAME I-ESIMO

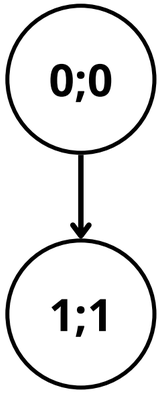
0;0

Situazione iniziale

Crediti Posseduti	0	1	2	3	4
Precedente	0	∞	∞	∞	∞
Attuale	∞	∞	∞	∞	∞

3	4
1	1
2	3
4	1

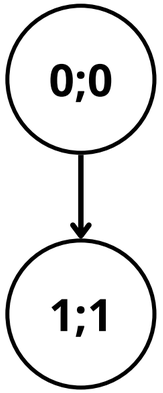
Primo passo



Crediti Posseduti	0	1	2	3	4
Precedente	0	∞	∞	∞	∞
Attuale	∞	1	∞	∞	∞

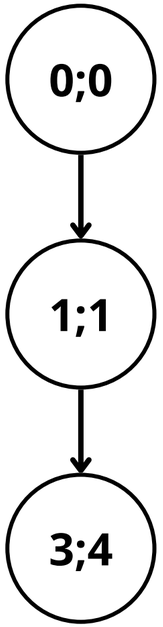
3	4
1	1
2	3
4	1

Inversione e reset



Crediti Posseduti	0	1	2	3	4
Precedente	∞	1	∞	∞	∞
Attuale	∞	∞	∞	∞	∞

3	4
1	1
2	3
4	1

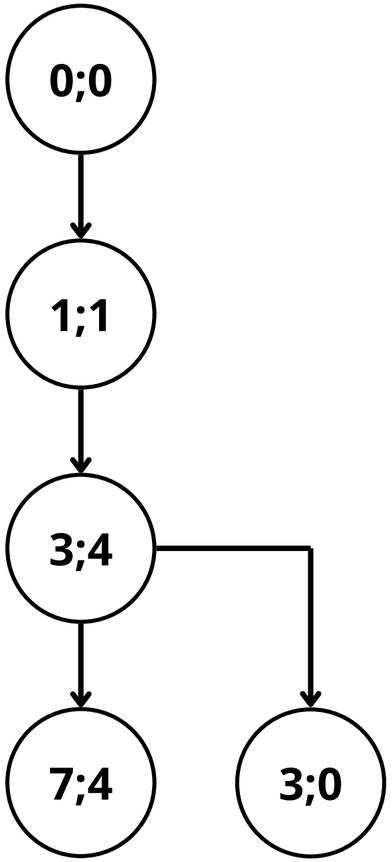


Secondo passo

Crediti Posseduti	0	1	2	3	4
Precedente	∞	1	∞	∞	∞
Attuale	∞	∞	∞	∞	3

3	4
1	1
2	3
4	1

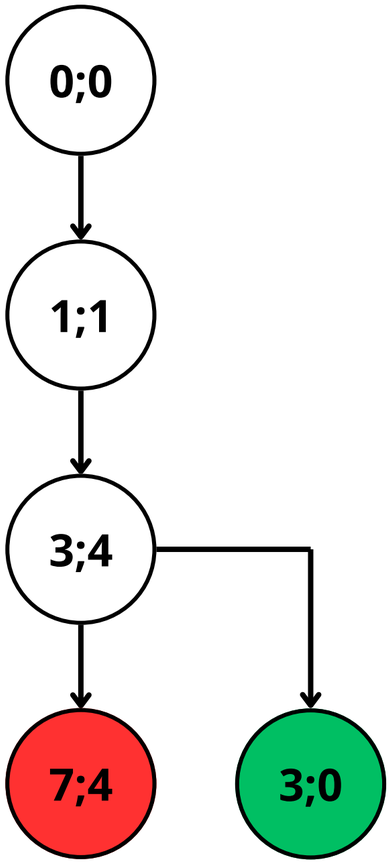
Terzo passo



Crediti Posseduti	0	1	2	3	4
Precedente	∞	∞	∞	∞	3
Attuale	3	∞	∞	∞	7

3	4
1	1
2	3
4	1

Finale



Crediti Posseduti	0	1	2	3	4
Attuale	3	∞	∞	∞	7

OUTPUT: 3

3	4
1	1
2	3
4	1

Complessità algoritmo

Complessità temporale = O(N*K):

```
for (int i = 0; i < N; i++) {
    ...
    for (int c = 0; c ≤ K; c++) {
        ...
    }
    ...
    for (int c = 0; c ≤ K; c++) {
        ...
    }
    ...
}
```

Complessità spaziale= O(K):

```
long long *dp_prev = (long long *)malloc((K + 1) *
sizeof(long long));

long long *dp_curr = (long long *)malloc((K + 1) *
sizeof(long long));
```

GRAZIE

Per l’attenzione